Q1 | Earliest-connection queries

Revised 5 October 2026

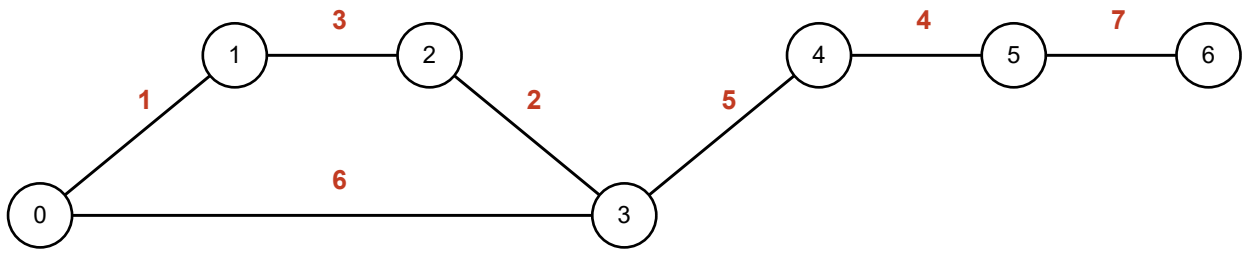
10 marks. Design an efficient solution for an undirected edge-insertion stream.

Problem statement

Initially, vertices 0, 1, ..., n-1 are isolated. Edges in the sequence L are inserted one at a time; the i-th edge is inserted at time i (1-based). After processing the stream, answer each pair (u, v) in Q with the earliest time at which the vertices became connected.

Return **0 when u = v**, and **-1 if the vertices never become connected**. Return one integer per pair, in the original order of Q. You may preprocess the stream and handle queries offline, but must not rerun a graph traversal separately for every query.

Example 1



Red edge labels show insertion times; n = 7.

```
n = 7
L = [(0,1),(2,3),(1,2),(4,5),(3,4),(0,3),(5,6)]
Q = [(0,4),(1,6),(0,2),(2,6),(5,3)]
Return: [5,7,3,7,5]
```

For (0,4), edge 5 first creates a path 0-1-2-3-4. Edge 6 later provides a shorter path, but does not change the earliest connection time. Vertex 6 remains isolated until edge 7.

Example 2: disconnected and identical vertices

```
n = 5
L = [(0,1),(1,2),(3,4)]
Q = [(0,2),(2,3),(4,4)]
Return: [2,-1,0]
```

Implementation and valid inputs

Use Q1.ipynb and implement **EarliestConnectionQuery.query(n, L, Q)**. Work on your own copy locally or in Google Colab. Keep the class and method signatures unchanged. Put implementation imports and any helper definitions before the local-test cell.

You may assume n is a non-negative integer, every vertex ID is in 0..n-1, and L contains no self-loops or repeated undirected edges. L and Q may be empty. Q may contain repeated pairs and pairs with identical endpoints. If n = 0, both L and Q are empty. The query count is independent of the edge count.

Only the Python Standard Library may be used. Hidden tests may contain up to **100,000 vertices, 500,000 edges and 100,000 queries**. The local tests include examples, small graphs checked against a brute-force oracle, edge cases and a deterministic scale test.

Supporting document: Q1.pdf

Explain your algorithm and why it returns the earliest connection time. Analyse stream processing, one query and total time for q queries. For a batch/offline algorithm, state its total batch cost and explain any amortised per-query figure; do not omit repeated preprocessing.

Use n for vertices, m for inserted edges and q for query pairs. State auxiliary data-structure space separately from input storage and the O(q) returned answer list. Include initialisation costs when isolated vertices are possible. Diagrams and examples are encouraged.

Submission and marking

Component	Marks
Code correctness and efficiency, with clear explanation/comments	8
Time complexity analysis	1
Space complexity analysis	1

Submit **Q1.ipynb and Q1.pdf** on Moodle, following the Project Summary. Restart the notebook session and run all cells in order before submission. The supplied TODOs are intentional; complete them without changing the tests.

END OF QUESTION